

Windows平台 FFmpeg D3D11VA硬件帧渲染到Avalonia的完整方案

核心思路

FFmpeg D3D11VA解码 → ID3D11VideoProcessor转换 → EGLImage共享 → OpenGL纹理 → Avalonia渲染

第一步：设备共享（最关键）

FFmpeg必须使用与Avalonia/ANGLE相同的D3D11设备，否则纹理无法共享。

正确流程：

1. 在 `OpenGLControlBase.OnOpenGLInit` 里通过EGL扩展获取ANGLE的D3D11设备指针
2. 用这个指针构建FFmpeg的硬件设备上下文

csharp

```
// 1. 获取ANGLE的D3D11Device指针
const int EGL_DEVICE_EXT = 0x322C;
const int EGL_D3D11_DEVICE_ANGLE = 0x33A1;

var fnGetDisplay = GetDelegate<EglGetCurrentDisplay>(gl, "eglGetCurrentDisplay");
var fnQueryDisplay = GetDelegate<EglQueryDisplayAttrib>(gl, "eglQueryDisplayAttribEXT");
var fnQueryDevice = GetDelegate<EglQueryDeviceAttrib>(gl, "eglQueryDeviceAttribEXT");

var display = fnGetDisplay();
fnQueryDisplay(display, EGL_DEVICE_EXT, out var eglDevice);
fnQueryDevice(eglDevice, EGL_D3D11_DEVICE_ANGLE, out var d3d11DevicePtr);

// 2. 用Vortice包装
_device = new ID3D11Device(d3d11DevicePtr);
_context = _device.ImmediateContext;
_videoDevice = _device.QueryInterface<ID3D11VideoDevice>();
_videoContext = _context.QueryInterface<ID3D11VideoContext>();

// 3. 注入FFmpeg（必须在OnOpenGLInit之后）
var hwDeviceContextBuffer =
    ffmpeg.av_hwdevice_ctx_alloc(AVHWDeviceType.AV_HWDEVICE_TYPE_D3D11VA);
var hwDeviceCtx = (AVHWDeviceContext*)hwDeviceContextBuffer->data;
var d3d11vaDeviceCtx = (AVD3D11VADeviceContext*)hwDeviceCtx->hwctx;
d3d11vaDeviceCtx->device = (FFmpeg.AutoGen.ID3D11Device*)d3d11DevicePtr;
ffmpeg.av_hwdevice_ctx_init(hwDeviceContextBuffer);
```

第二步：解码格式

必须使用 `AV_HWDEVICE_TYPE_D3D11VA` 而不是 `DXVA2`，解码得到的帧格式为 `AV_PIX_FMT_D3D11`：

csharp

```
// data[0] = ID3D11Texture2D* (DECODER纹理)
// data[1] = ArrayIndex (纹理数组索引)
var decoderTexPtr = (IntPtr)frame->data[0];
var arrayIndex = (int)(nint)frame->data[1];
```

第三步：VideoProcessor转换

DECODER纹理无法直接被Shader采样，需要通过 `ID3D11VideoProcessor` 在GPU内转换为RGBA格式。

输出纹理必须带以下flags：

csharp

```
var texDesc = new Texture2DDescription
{
    Format = Format.R8G8B8A8_UNorm,
    BindFlags = BindFlags.ShaderResource | BindFlags.RenderTarget,
    MiscFlags = ResourceOptionFlags.Shared | ResourceOptionFlags.SharedNthandle,
    // 其他字段用实际视频分辨率
};
```

VideoProcessorInputView使用 `ArraySlice` 指定纹理数组索引：

csharp

```
var ivDesc = new VideoProcessorInputViewDescription
{
    FourCC = 0,
    ViewDimension = VideoProcessorInputViewDimension.Texture2D,
    Texture2D = new Texture2DVideoProcessorInputView
    {
        MipSlice = 0,
        ArraySlice = (uint)arrayIndex
    }
};
```

VideoProcessor尺寸必须与实际纹理尺寸一致，建议第一帧时动态初始化。

第四步：EGLImage共享

使用 `EGL_ANGLE_image_d3d11_texture` 扩展，正确常量值如下（来自ANGLE源码）：

csharp

```
const uint EGL_D3D11_TEXTURE_ANGLE = 0x3484;
const uint EGL_D3D11_TEXTURE_ARRAY_SLICE_ANGLE = 0x3493;
const uint EGL_NONE = 0x3038;

int[] attribs =
[
```

```

        (int)EGL_D3D11_TEXTURE_ARRAY_SLICE_ANGLE, 0,
        (int)EGL_NONE
    ];

    var eglImage = _eglCreateImage(
        _eglDisplay,
        eglContext, // 必须传当前EGL context, 不能传IntPtr.Zero
        EGL_D3D11_TEXTURE_ANGLE,
        _outputTex.NativePointer, // 直接传D3D11纹理指针
        attribs);

```

此操作必须在GL线程上执行（即OnOpenGLRender内）。

第五步：GL纹理绑定

EGLImage创建后绑定到GL纹理，必须设置纹理过滤参数，否则采样返回全黑：

csharp

```

gl.BindTexture((int)GL_TEXTURE_2D, (int)_glTexture);
_eglEGLImageTargetTexture2D(GL_TEXTURE_2D, eglImage);
_eglDestroyImage(_eglDisplay, eglImage); // 绑定后立即销毁EGLImage, GL纹理仍有效

// 必须设置, 否则全黑
gl.TexParameteri((int)GL_TEXTURE_2D, GLConsts.GL_TEXTURE_MIN_FILTER, GLConsts.GL_LINEAR);
gl.TexParameteri((int)GL_TEXTURE_2D, GLConsts.GL_TEXTURE_MAG_FILTER, GLConsts.GL_LINEAR);
gl.TexParameteri((int)GL_TEXTURE_2D, 0x2802, 0x812F); // GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE
gl.TexParameteri((int)GL_TEXTURE_2D, 0x2803, 0x812F); // GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE

```

第六步：渲染

使用硬编码顶点的shader渲染全屏四边形，渲染到Avalonia提供的framebuffer：

csharp

```

// vertex shader: 硬编码全屏四边形和UV
// fragment shader: 采样uTexture

// 渲染时绑定Avalonia的FBO
gl.BindFramebuffer(GLConsts.GL_FRAMEBUFFER, framebuffer); // framebuffer来自OnOpenGLRender参数
gl.Viewport(0, 0, (int)Bounds.Width, (int)Bounds.Height);
gl.DrawArrays(GLConsts.GL_TRIANGLES, 0, 6);

```

关键踩坑点汇总

问题	原因	解决方式
CreateVideoProcessorInputView E_INVALIDARG	FFmpeg和ANGLE使用了不同的D3D11设备	让FFmpeg使用ANGLE的设备指针
eglCreatImageKHR返回null	EGL常量值错误， EGL_D3D11_TEXTURE_ARRAY_SLICE_ANGLE 应为 0x3493	从ANGLE源码确认常量值
GL纹理采样全黑	EGLImage绑定后未设置纹理过滤参数	绑定后立即设置MIN/MAG filter和WRAP参数
画面不显示	渲染到了默认FBO(0)而不是Avalonia的FBO	使用OnOpenGLRender传入的framebuffer参数
av_hwdevice_ctx_init崩溃	设备指针注入时序问题，OnOpenGLInit尚未调用	用TaskCompletionSource等待GL初始化完成后再初始化FFmpeg
AVFrame->data[0]为null	使用了DXVA2而不是D3D11VA	改用AV_HWDEVICE_TYPE_D3D11VA，帧格式为 AV_PIX_FMT_D3D11

其他平台可能的总结

Android平台的优势

Android原生就是EGL + OpenGL ES环境，Avalonia在Android上同样使用EGL，**不需要像Windows那样绕道ANGLE**。整个共享机制更简洁。

Android的硬件解码路径

FFmpeg在Android上的硬件解码后端是 `MediaCodec`，解码得到的帧格式为 `AV_PIX_FMT_MEDIACODEC`。

与Windows D3D11不同，MediaCodec的输出直接就是 **OpenGL ES纹理**，格式为 `GL_TEXTURE_EXTERNAL_OES`，不需要VideoProcessor转换步骤。



关键差异

纹理类型不同

Android用的是 `GL_TEXTURE_EXTERNAL_OES` 而不是 `GL_TEXTURE_2D`，shader需要特殊扩展：

glsl

```
#version 300 es
#extension GL_OES_EGL_image_external_essl3 : require
precision mediump float;
in vec2 vTexCoord;
uniform samplerExternalOES uTexture; // 不是sampler2D
out vec4 fragColor;
void main()
{
    fragColor = texture(uTexture, vTexCoord);
}
```

设备共享问题

Android上不存在D3D11设备共享的问题，但需要把 **Avalonia的EGL Context** 传给MediaCodec的SurfaceTexture，让解码输出直接写入这个Context下的纹理。

csharp

```
// 需要在Avalonia的GL线程上创建SurfaceTexture
// 然后把Surface传给FFmpeg的MediaCodec
```

颜色矩阵变换

MediaCodec输出通常是YUV颜色空间，通过 `GL_TEXTURE_EXTERNAL_OES` 采样时驱动会自动做YUV→RGB转换，但需要正确处理颜色矩阵：

glsl

```
// SurfaceTexture提供变换矩阵
uniform mat4 uTransformMatrix;

void main()
{
    vec2 transformedCoord = (uTransformMatrix * vec4(vTexCoord, 0.0, 1.0)).xy;
    fragColor = texture(uTexture, transformedCoord);
}
```

主要挑战

1. SurfaceTexture与Avalonia GL Context绑定

这是Android上最难的部分，和Windows的设备共享问题类似：

必须在Avalonia的GL线程上创建GLuint纹理

→ 用这个纹理创建SurfaceTexture

→ 把SurfaceTexture.getSurface()传给MediaCodec

→ MediaCodec解码后调用updateTexImage()更新纹理

2. FFmpeg.AutoGen的MediaCodec支持

FFmpeg.AutoGen对MediaCodec的绑定不如D3D11完整，可能需要部分P/Invoke直接调用Android API。

3. Avalonia Android的OpenGL接入

Avalonia在Android上是否完整支持 `OpenGLControlBase` 还需要确认，可能需要用 `NativeControlHost` 或其他方式接入。

其他平台简述

平台	硬件解码后端	共享机制	难度
Windows	D3D11VA	ANGLE EGL + D3D11纹理共享	已解决
Android	MediaCodec	GL_TEXTURE_EXTERNAL_OES + SurfaceTexture	中等
Linux	VAAPI	DMA-BUF + EGL Image	中等
macOS/iOS	VideoToolbox	Metal纹理 → OpenGL共享	较难

总结

Android平台的路径实际上比Windows更简单，因为：

- 1. 不需要VideoProcessor转换步骤
- 2. 不需要EGLImage共享，MediaCodec直接输出GL纹理
- 3. 原生EGL环境，不需要绕道ANGLE